Protocolo Experimental

Tema 4 — APIs remotas e processamento assíncrono

Demonstração com a aplicação "Locais IPMA" (Kotlin)

Grupo 8

Tiago Gomes · Rodrigo Ferreira · Gonçalo Silva

Universidade da Beira Interior — Engenharia Informática · Covilhã, 3 de novembro de 2026

Repositório: `https://github.com/tiagogit1/ApresentacaoPDM`

---

1 Enquadramento teórico

Uma aplicação móvel obtém dados de um servidor através de uma API remota. O pedido demora tempo e pode falhar, por isso tem de ser feito sem bloquear a interface e com tratamento de erros. Os conceitos usados na demonstração são os seguintes.

1.1 HTTP, REST e JSON

O HTTP é um protocolo de pedido–resposta sem estado: o cliente envia um método e um caminho; o servidor responde com um código de estado (2xx sucesso, 4xx erro do cliente, 5xx erro do servidor) e um corpo. Numa API REST os recursos são identificados por URI (por exemplo, `/open-data/distrits-islands.json`) e o `GET` é *seguro* e *idempotente*: repetir o pedido não altera o servidor, logo pode repetir-se após uma falha (o `POST` não). Os dados vêm normalmente em JSON, que a *serialização* converte em objetos. Em Kotlin usa-se kotlinx.serialization (classes `@Serializable`, sem reflexão); em Java, Gson ou Moshi. Campos que a classe não declara exigem `ignoreUnknownKeys`, senão a conversão falha. Em Android o tráfego deve usar HTTPS e o manifesto precisa da permissão `INTERNET`.

1.2 OkHttp e Retrofit

O OkHttp é o cliente HTTP: gere ligações, timeouts (`connectTimeout`, `readTimeout`, `writeTimeout`, `callTimeout`), interceptores e cache HTTP. O Retrofit assenta no OkHttp: a API descreve-se como uma interface anotada (`@GET`, `@Path`, `@Query`) e um conversor transforma o JSON em objetos.

1.3 Execução em segundo plano

A main thread desenha a interface (a 60 Hz, cerca de 16,7 ms por *frame*). Um pedido de rede feito nela congela a interface: o Android lança `NetworkOnMainThreadException` e, se a aplicação deixar de responder durante cerca de 5 s, mostra o diálogo ANR. Em Kotlin usam-se **coroutines**: uma função `suspend` pausa sem bloquear a thread; o `viewModelScope` cancela o trabalho quando o ViewModel é destruído e os *dispatchers* (`Main`, `IO`, `Default`) escolhem a thread. Com o Retrofit, uma função `suspend` é *main-safe*. A alternativa em **Java** é o `ExecutorService`, que executa a tarefa num *pool* de threads; o resultado volta à main thread com `postValue()` do `LiveData` ou com um `Handler`.

1.4 Estado, erros e cache

O **ViewModel** guarda o estado do ecrã e sobrevive à rotação; em Kotlin o estado é um `StateFlow` (em Java, `LiveData`). Cada falha tem a sua resposta: sem rede (`IOException`), timeout (`InterruptedIOException`, subclasse da anterior e por isso testada primeiro) e erro HTTP (`HttpException`, 4xx/5xx). A aplicação guarda a última resposta e os favoritos em **SharedPreferences**, mostra-os ao abrir e atualiza-os quando a rede responde; um indicador circular (primeiro carregamento) ou linear (atualização com a lista visível) informa o utilizador.

2 Como testar a aplicação

A aplicação "Locais IPMA" obtém da API aberta do IPMA a lista dos 35 locais de previsão (capitais de distrito e ilhas), apresenta-a, guarda no dispositivo a última resposta e os favoritos, e continua a funcionar sem rede.

2.1 Objetivos

1. Observar um GET real a uma API REST e a conversão do JSON em objetos Kotlin.
2. Comprovar que a rede não pode ser usada na main thread e que, com coroutines, a interface se mantém fluida.
3. Verificar o comportamento sem rede (dados guardados, mensagem de erro) e a recuperação.
4. Confirmar que o estado sobrevive à rotação do ecrã e que não há pedidos repetidos.

2.2 Configuração

Item	Descrição
Computador	Android Studio recente (AGP 8.13, compileSdk 36, Java 17 como alvo) e internet.
Dispositivo	Emulador ou telemóvel com Android 8.0 (API 26) ou superior. O teste T5 só é possível no emulador.
Projeto	Kotlin 2.2, Jetpack Compose, Retrofit 3.0, OkHttp 4.12, kotlinx.serialization 1.9, ViewModel e StateFlow.
Repositório	`https://github.com/tiagogit1/ApresentacaoPDM` (acesso só de leitura; as alterações de T2 e T6 fazem-se numa cópia local).
API	`https://api.ipma.pt/open-data/distrits-islands.json` (apenas GET, sem chave).
Ferramentas	Network Inspector (App Inspection), Logcat e um navegador web.

A API é aberta e a aplicação não usa palavras-passe, tokens nem chaves; o repositório não contém palavras-passe, tokens, chaves de assinatura nem dados pessoais.

2.3 Dados de entrada

- **Resposta da API:** JSON de `GET /open-data/distrits-islands.json`, com `owner`, `country` e a lista `data` de 35 locais; a aplicação usa `globalIdLocal`, `local`, `latitude` e `longitude`. Sem parâmetros nem autenticação.
- **Estado inicial:** dados da aplicação limpos (sem lista nem favoritos guardados).
- **Ações do utilizador:** ☆ em Castelo Branco e na Guarda; *Atualizar*; *Tentar de novo*.
- **Condições de rede:** normal; modo de avião (T3); EDGE com sinal fraco no emulador (T5).
- **Alterações ao código:** T2 (pedido síncrono na main thread) e T6 (`callTimeout` de 1 ms; caminho `open-data/nao-existe.json`).

2.4 Preparação

1. Clonar o repositório (`git clone https://github.com/tiagogit1/ApresentacaoPDM`) e abrir o projeto no Android Studio (**File** → **Open**); aguardar o Gradle.
2. Executar a aplicação (Run) no emulador ou telemóvel e confirmar que abre.
3. Abrir o **Network Inspector** (View → Tool Windows → App Inspection) no processo `pt.ubi.locais` e o **Logcat** com o filtro `package:mine`.
4. No dispositivo: Definições → Apps → Locais IPMA → Armazenamento → Limpar dados.

Regras. Fazer os testes pela ordem indicada (T3–T5 dependem dos dados obtidos em T1). Depois de T2 e de T6 repor o código original com `git checkout ..` **Contagem de pedidos:**

contar só as linhas com estado 200 e tamanho superior a 0 (o Network Inspector pode mostrar uma linha vazia adicional por pedido).

2.5 Procedimento e resultados esperados

T1 — Primeiro carregamento (GET real e JSON).

1. Confirmar que os dados foram limpos (passo 4) e abrir a aplicação com o Network Inspector aberto.
2. Enquanto o indicador circular está visível, puxar a barra de notificações e voltar: o sistema e a aplicação respondem?
3. No Network Inspector, selecionar `distrits-islands.json` e anotar método, código de estado e duração; contar os locais da lista (scroll até ao fim).
4. No browser, abrir o URL da API, localizar Castelo Branco e comparar com a classe `Local` em `Rede.kt`: que campos declara? Em que formato vêm latitude e longitude?

Esperado: indicador circular e depois a lista ordenada alfabeticamente, com "Atualizado em…"; GET 200 com duração de dezenas a centenas de ms (só o primeiro pedido costuma ser mais lento); 35 locais; a interface responde. A classe `Local` declara 4 campos e ignora os restantes (`ignoreUnknownKeys = true`); as coordenadas vêm em texto.

T2 — Rede na main thread (altera o código). No fim de `onCreate` de `MainActivity.kt`, acrescentar temporariamente:

```
try {
    okhttp3.OkHttpClient().newCall(
        okhttp3.Request.Builder()
            .url("https://api.ipma.pt/open-data/distrits-islands.json")
            .build()
    ).execute()          // pedido síncrono na main thread - proibido
} catch (e: Exception) {
    android.util.Log.e("TESTE", "Pedido na main thread", e)
}
```

Executar e ver o Logcat com o filtro `TESTE`; depois remover o bloco. *Esperado:* `android.os.NetworkOnMainThreadException`; o pedido nem chega a sair e a aplicação não fecha (a exceção é apanhada). Na versão normal, `obterLocais()` é `suspend`: o pedido corre nas threads do OkHttp.

T3 — Favoritos guardados e funcionamento sem rede.

1. Tocar em ☆ em Castelo Branco e na Guarda; observar a posição dos locais e o Network Inspector.
2. Anotar a hora de "Atualizado em…", ativar o modo de avião, fechar a aplicação pelos recentes e voltar a abri-la.

Esperado: os dois locais passam a ★ e sobem para o topo, sem pedidos de rede. Sem rede, a lista e os favoritos aparecem (lidos de SharedPreferences) com a caixa "Sem ligação à internet." e o botão "Tentar de novo"; a hora é a mesma de antes.

T4 — Recuperação quando a rede volta. Desligar o modo de avião e tocar em "Tentar de novo". *Esperado:* barra linear no topo sem esconder a lista; a caixa de erro desaparece; a hora muda para a atual; novo GET 200.

T5 — Rede lenta e rotação (só no emulador). Ativar **⋯ (Extended controls)** → **Cellular** → **Network type**: EDGE, **Signal strength**: Poor, com o Wi-Fi do emulador desligado. (Com a rede normal os pedidos demoram cerca de 20 ms, terminam antes de a rotação os interromper e dois toques rápidos podem dar dois pedidos.)

1. Tocar em *Atualizar*; enquanto a barra linear está visível, fazer scroll e marcar/desmarcar um favorito; anotar a duração e comparar com T1.
2. Tocar em *Atualizar* e, de imediato, rodar o dispositivo; contar os pedidos. Rodar mais duas vezes sem atualizar e contar de novo.
3. Tocar duas vezes seguidas e rapidamente em *Atualizar*.
4. No fim, repor **Network type**: Full e ligar o Wi-Fi.

Esperado: o pedido demora mais do que em T1 (centenas de ms a alguns segundos; acima de 20 s surge "O servidor demorou demasiado a responder."), mas o scroll e os favoritos respondem. A rotação mantém o estado (ViewModel) e dá 1 pedido; rodar sem atualizar dá 0; dois toques rápidos dão 1 pedido (o botão fica desativado).

T6 — Erros forçados (altera o código, uma alteração de cada vez).

1. *Timeout:* em `Rede.kt`, mudar `.callTimeout(20, TimeUnit.SECONDS)` para `.callTimeout(1, TimeUnit.MILLISECONDS)`; tocar em *Atualizar*; anotar a mensagem; repor.
2. *Erro HTTP:* mudar o caminho do `@GET` (interface `IpmaApi`) para `"open-data/nao-existe.json"`; tocar em *Atualizar*; anotar a mensagem e o código no Network Inspector; repor.

Esperado: timeout → "O servidor demorou demasiado a responder." (`InterruptedIOException`, testada antes de `IOException`); caminho errado → "O servidor respondeu com erro (HTTP 4xx).", normalmente 404 ou 403. Nos dois casos a lista guardada mantém-se visível.

2.6 Métricas e registo

Métricas: método e código HTTP e duração do pedido (Network Inspector); número de pedidos (Network Inspector, com a regra de contagem acima); número de locais (contagem na lista); exceção (Logcat); mensagem e hora "Atualizado em…" (ecrã); a interface responde de imediato durante o pedido, sem travar (observação).

Tabela 1. Registo dos testes (valores esperados).

Teste	Grandeza	Esperado
T1	Método / código HTTP	GET / 200
T1	Duração do pedido (ms)	≈ 10–800
T1	N.º de locais	35
T1	Tipo de latitude no JSON	texto
T1	Interface responde?	Sim
T2	Exceção registada	`NetworkOnMainThreadException`
T2	O pedido chegou ao servidor?	Não
T3	Pedidos ao marcar ★	0
T3	Mensagem sem rede	Sem ligação à internet.
T3	Lista visível / hora igual?	Sim / Sim
T4	Indicador / hora mudou?	barra linear / Sim
T4	Código do novo pedido	200
T5	Duração com EDGE (ms)	> duração de T1
T5	Scroll fluido durante o pedido?	Sim
T5	Pedidos: Atualizar + rotação	1
T5	Pedidos: rodar 2× sem atualizar	0
T5	Pedidos: 2 toques rápidos	1
T6	Mensagem com `callTimeout` 1 ms	O servidor demorou demasiado…
T6	Código HTTP com caminho errado	4xx (404/403)
T6	Lista visível nos dois casos?	Sim

No fim: repor o código (`git checkout .`), a rede do emulador (Network type: Full, Wi-Fi ligado) e desligar o modo de avião.

3 Excertos em Java

A aplicação foi desenvolvida em Kotlin. Quando uma tecnologia é só de Kotlin, a alternativa em Java é a da tabela; os excertos seguintes mostram o pedido e o ViewModel em Java (versão simplificada; a completa, com `Handler`, está em `excerto-java/LocaisViewModelJava.java`, no repositório).

Elemento	Kotlin (aplicação)	Java
Pedido Retrofit	`suspend fun obterLocais()`	`Call<RespostaLocais>` com `execute()` ou `enqueue()`
Conversão JSON	kotlinx.serialization	Gson ou Moshi
Execução em fundo	coroutines (`viewModelScope`)	`ExecutorService`
Voltar à main thread	automático	`postValue()` ou `Handler`
Estado observável	`StateFlow`	`LiveData`
Interface	Jetpack Compose	Views XML + `RecyclerView`

Interface Retrofit e DTO (Gson).

```
interface IpmaApi {
    @GET("open-data/distrits-islands.json")
    Call<RespostaLocais> obterLocais();
}
class RespostaLocais { List<Local> data; }
class Local { int globalIdLocal; String local;
              String latitude; String longitude; }
```

ViewModel com ExecutorService e LiveData.

```
public class LocaisViewModelJava extends ViewModel {
  private final ExecutorService executor =
      Executors.newFixedThreadPool(2);
  private final MutableLiveData<List<Local>> locais =
      new MutableLiveData<>();
  private final MutableLiveData<Boolean> aCarregar =
      new MutableLiveData<>(false);
  private final MutableLiveData<String> erro =
      new MutableLiveData<>();

  void carregar() {                    // main thread
    if (Boolean.TRUE.equals(aCarregar.getValue())) return;
    aCarregar.setValue(true);
    executor.execute(() -> {           // thread do pool
      try {
        Response<RespostaLocais> r = Rede.api.obterLocais().execute();
        if (r.isSuccessful() && r.body() != null)
          locais.postValue(r.body().data);
        else
          erro.postValue("O servidor respondeu com erro (HTTP "
                         + r.code() + ").");
      } catch (InterruptedIOException e) {   // timeout: primeiro
        erro.postValue("O servidor demorou demasiado a responder.");
      } catch (IOException e) {
        erro.postValue("Sem ligação à internet.");
      } finally {
        aCarregar.postValue(false);
      }
    });
  }

  @Override protected void onCleared() { executor.shutdownNow(); }
}
```